

Sol Unix Lab

Python: List Current Directory

```
1 import os
2 import sys
3
4 try:
5     # 1. Get the current working directory
6     cwd = os.getcwd()
7     print(f"Current Directory: {cwd}")
8     print("-" * 20)
9
10    # 2. Get the list of entries in that directory
11    entries = os.listdir(cwd)
12    # 3. Print each entry
13    print("Directory Contents:")
14    if not entries:
15        print("(This directory is empty)")
16    else:
17        for entry in entries:
18            print(entry)
19
20 except OSError as e:
21    # Handle potential errors like permission denied
22    print(f"Error: Could not access directory. {e}", file=sys.stderr)
23    sys.exit(1)
```

Shell: Calculate Average Score (awk)

```
1 awk '{
2     sum[$1] += $2;
3     count[$1]++;
4 }
5 END {
6     for (user in sum) {
7         print user, sum[user]/count[user];
8     }
9 }' scores.txt
```

SET 2

Python: Find .txt Files in a Directory

```
1 import os
2 import sys
3
4 def find_txt_files(directory_path):
5     # Check if the correct number of arguments is provided
6     if len(sys.argv) != 2:
7         print(f"Usage: python {sys.argv[0]} <directory_path>", file=sys.stderr)
8         sys.exit(1)
9
10    directory_path = sys.argv[1]
11
12    # Check if the provided path is a valid directory
13    if not os.path.isdir(directory_path):
14        print(f"Error: Path '{directory_path}' is not a valid directory.",
15              ↪ file=sys.stderr)
16        sys.exit(1)
17
18    try:
19        # Get all entries in the directory
20        entries = os.listdir(directory_path)
21        found_files = False
22        # Loop through entries and check if they are files ending in .txt
23        for entry in entries:
24            # We only care about files, so we check 'isfile'
25            full_path = os.path.join(directory_path, entry)
26            if os.path.isfile(full_path) and entry.endswith(".txt"):
27                print(entry)
28                found_files = True
29        if not found_files:
30            print("No .txt files found.")
31
32    except OSError as e:
33        print(f"Error reading directory: {e}", file=sys.stderr)
34        sys.exit(1)
35
36    if __name__ == "__main__":
37        find_txt_files(sys.argv)
```

Shell: Prefix “WARNING” Lines with “FIXME:” (sed)

```
1 sed -i '/WARNING/ s/^/FIXME: /' system.log
```

Python: Sum Command-Line Integers

```

1  import sys
2
3  def sum_arguments():
4      # Get all arguments *after* the script name
5      args = sys.argv[1:]
6
7      # If no arguments are provided, print 0
8      if not args:
9          print(0)
10         return
11
12     total_sum = 0
13     try:
14         # Loop through each argument, convert to int, and add to sum
15         for arg in args:
16             total_sum += int(arg)
17         print(total_sum)
18
19     except ValueError:
20         # Handle the case where a non-integer is passed
21         print(f"Error: All arguments must be integers.", file=sys.stderr)
22         sys.exit(1) # Exit with an error code
23     except Exception as e:
24         print(f"An unexpected error occurred: {e}", file=sys.stderr)
25         sys.exit(1)
26
27 if __name__ == "__main__":
28     sum_arguments()

```

Shell: Total Salary by Department (awk)

```

1  awk -F, 'NR > 1 {
2      sum[$2] += $3
3  }
4  END {
5      for (dept in sum) {
6          print dept, sum[dept]
7      }
8  }' employees.csv

```

Python: Uppercase a File with argparse

```
1 import argparse
2 import sys
3
4 def main():
5     # 1. Create the parser
6     parser = argparse.ArgumentParser(
7         description="Convert a file's contents to uppercase."
8     )
9
10    # 2. Add arguments
11    parser.add_argument(
12        "input_file",
13        help="The path to the input file to read."
14    )
15    parser.add_argument(
16        "-o", "--output",
17        help="The optional output file path. (Default: prints to console)"
18    )
19
20    # 3. Parse the arguments
21    args = parser.parse_args()
22
23    try:
24        # Read the input file
25        with open(args.input_file, 'r') as f:
26            content = f.read()
27            uppercase_content = content.upper()
28
29        # Check if an output file was specified
30        if args.output:
31            # Write to the output file
32            with open(args.output, 'w') as f:
33                f.write(uppercase_content)
34            print(f"Successfully wrote uppercase content to {args.output}")
35        else:
36            # Print to the console (stdout)
37            print(uppercase_content)
38
39    except FileNotFoundError:
40        print(f"Error: Input file '{args.input_file}' not found.", file=sys.stderr)
41        sys.exit(1)
42    except Exception as e:
43        print(f"An error occurred: {e}", file=sys.stderr)
44        sys.exit(1)
```

```
45
46 if __name__ == "__main__":
47     main()
```

Shell: Find Whole Word “cat” (grep)

This grep command finds the word *cat* case-insensitively.

- -i: Makes the search case-insensitive.
- -w: Matches only whole words (not catfish or catalog).
- -n: Prepends the line number to each matching line.

```
1 grep -iwn 'cat' animals.txt
```
